BHIMA ASTRA

GigSheild: AI-Powered Parametric Insurance for Gig Delivery Workers

Guidewire DEVTrails 2026 - Phase 1 Submission

Table of Contents

	Section Title	Page No.
1	Introduction and Problem Understanding	
2	Our Solution Overview	
3	Why This Model: Design Decisions	
4	End to End System Flow	
5	Parametric Trigger System	
6	AI and Machine Learning Architecture	
7	Adversarial Defense and Anti-Spoofing Strategy	
8	Multi-Agent System Design	
9	Financial Model	
10	Stakeholder Benefits	
11	Advantages of Our Model	
12	Limitations and Future Improvements	
13	Technology Stack	
14	Final Conclusion	
15	Appendix	

1. Introduction and Problem Understanding

Gig delivery workers earn on a per-order basis, making their income highly volatile and directly dependent on external conditions such as weather, pollution, and platform availability. Despite this vulnerability, they lack access to structured income protection, as traditional insurance models are incompatible with their daily earning cycles.

BHIMA ASTRA introduces an AI-driven parametric insurance model that uses real-time environmental and operational data to detect disruption events and automatically trigger predefined payouts. Instead of claim-based assessment, the system relies on objective thresholds such as rainfall intensity, temperature, and air quality levels.

By combining predictive risk modeling with automated payout execution, the solution delivers immediate financial support during income disruptions. Designed with weekly premiums and fixed payout structures, it ensures affordability, transparency, and operational scalability for gig workers.

The Problem at Scale

Metric	Figure	Significance
Gig workers in India by 2030	23.5 Million	Rapidly growing unprotected workforce [1][2]
Q-commerce India market size (2025-26)	Rs. 64,000 Crore	Platform economy scale [3][4]
Gig workers with zero savings today	90%	Extreme financial vulnerability [5]
Monthly income lost during disruptions	20-30%	Directly insurable risk [6][7]
Delivery partners on Blinkit/Zepto/Instamart	3 Million+	Target addressable market [8]
Structured income protection available	None	The gap BHIMA ASTRA fills

These six data points define the gap this system fills: millions of workers, zero protection, and a measurable and insurable risk. [1]-[8]

Persona

USER PERSONA CONTENT

Core Profile

Name: Vishnu Kumar

Age and gender: 26, Male

Job: Gig Delivery Partner

Income range: ₹6,000 – ₹9,000 per week

Location: Hyderabad, India

Background

Short bio:
Vishnu is a full-time gig delivery worker who depends entirely on daily orders for his income. His earnings fluctuate based on external conditions such as weather, traffic, and platform availability. With minimal savings and high daily expenses, even a single disrupted day significantly impacts his financial stability.

Lifestyle and values:
- Works 8–10 hours daily, often during peak demand times
- Values consistent income over high but uncertain earnings
- Prefers practical, low-cost solutions
- Relies heavily on mobile apps for work and payments
- Focused on meeting daily financial needs (fuel, rent, food)

Professional context:
- Works across Q-commerce platforms
- Income depends on number of deliveries completed
- Faces reduced orders during rain, heatwaves, or disruptions
- No employer-provided insurance or income protection
- Operates in a highly competitive and uncertain environment

User Insights

Goals:
Short-term goals:
- Maintain consistent daily earnings
- Complete maximum deliveries per shift
- Avoid income loss during disruptions

Long-term goals:
- Build emergency savings
- Achieve financial stability
- Move towards a more secure livelihood

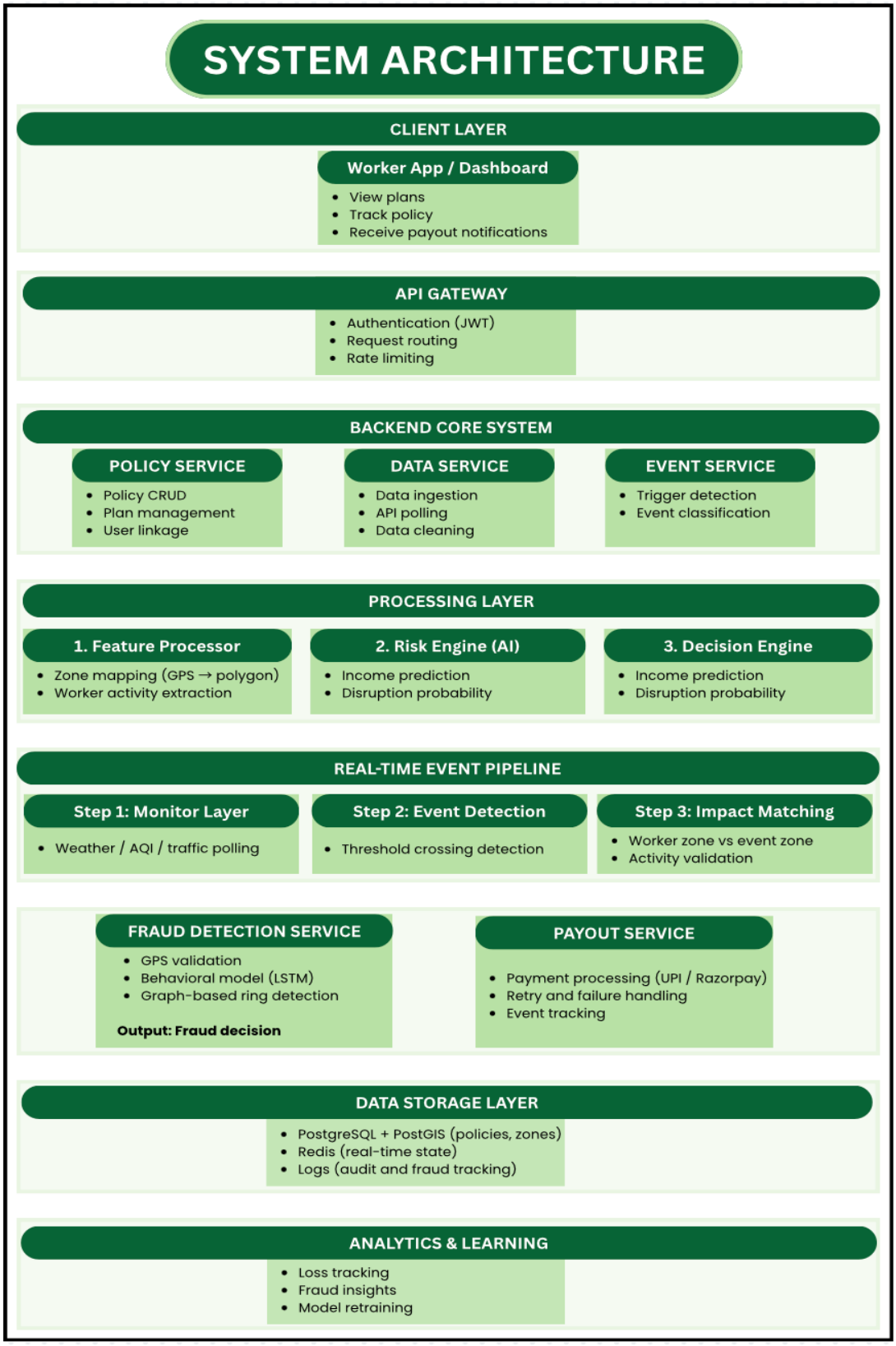
Key motivations:
- Immediate daily cash flow
- Stable and predictable income
- Low-cost financial support
- Quick and effortless solutions

Challenges:
Practical challenges:
- Income drops during rain, heatwaves, and pollution
- High fuel and maintenance expenses
- No alternative income source
- No access to suitable insurance products

Emotional challenges:
- Stress due to unpredictable earnings
- Pressure to meet daily expenses
- Frustration when unable to work

System challenges:
- Dependence on external factors
- No structured income protection
- Limited awareness of financial tools

2. Our Solution Overview



What the System Does

BHIMA ASTRA is an AI-driven parametric income protection platform designed exclusively for gig delivery workers operating across Q-commerce platforms in India, including **Swiggy Instamart, BigBasket Now, Flipkart Minutes, Amazon Now, FreshToHome Express, Zepto,** and **Blinkit** [1]-[7]. It addresses one precisely defined problem: when external disruptions such as extreme rainfall, heatwaves, severe air pollution, floods, platform outages, or social disruptions such as curfews and strikes prevent a worker from completing deliveries, the worker loses income with no safety net.

The system detects those disruptions in real time, evaluates them against predefined thresholds, and automatically transfers a fixed payout to the worker with no claim submission, no adjuster, and no delay.

The policy is structured on a weekly cycle, matching the income rhythm of gig workers. Workers select from three plan tiers: **Basic (Rs. 49/week), Standard (Rs. 79/week), or Premium (Rs. 119/week)**, each with predefined payout levels and a maximum of two covered events per week. Premiums are not uniform: the AI layer personalises each worker's premium within their chosen plan based on their delivery zone, seasonal disruption risk, historical event frequency, and delivery volume. The worker owns their individual policy independently of which platform they work on.

Key Components

Component	Role in System	Section Reference
Worker Registration and Profile	Captures platform, city, zone, vehicle type, shift hours, experience	Section 4 Step 1
AI Risk Engine	Random Forest (income prediction) + XGBoost (disruption probability)	Section 6
Weekly Premium Generator	Converts expected loss into personalised weekly premium per plan tier	Section 6.3
Real-Time Monitoring Layer	Continuously ingests weather, AQI, traffic, and platform status APIs	Section 4 Step 5
Parametric Trigger Engine	Evaluates multi-level thresholds (IMD/CPCB) and composite disruption score	Section 4 Step 7
Manager Intelligence Layer	Human-verified detection of curfews, protests, and zone shutdowns	Section 8 D3
Payout Execution Engine	Automatically transfers predefined payout when trigger fires	Section 4 Step 8

Component	Role in System	Section Reference
Fraud Detection Layer	GPS validation, anomaly detection, peer comparison, duplicate prevention	Section 9
Analytics Dashboard	Worker earnings view plus insurer loss ratio and predictive analytics	Section 10

Platform and Coverage Scope

The system covers income loss arising from five categories of external disruption, each with predefined parametric thresholds:

Disruption Category	Trigger Standard	Payout Levels
Heavy Rainfall	IMD: >= 64.5 mm (L1), >= 115.6 mm (L2), >= 204.5 mm (L3)	Rs. 300 / Rs. 600 / Rs. 1,200
Extreme Heat / Heatwave	IMD: >= 40 deg C (L1), >= 45 deg C (L2)	Rs. 300 / Rs. 600
Severe Air Pollution	CPCB AQI: >= 300 (L1), >= 400 (L2)	Rs. 300 / Rs. 500
Flood / Zone Shutdown	Flood alert flag or zone shutdown flag = 1	Plan-level payout
Platform Outage / Curfew	Platform API outage flag or manager-verified social disruption	Plan-level payout

Coverage is strictly limited to income loss only. Health, life, accident, and vehicle repair claims are explicitly excluded in all plan tiers.

Deployment Model

The system operates as a standalone insurance platform. Delivery platforms (Blinkit, Zepto, Swiggy Instamart, and others) act as distribution partners, promoting the product within their worker-facing apps and earning a **5-10% commission per active policy**. This is a **bancassurance-style model** that requires no insurance liability from the platform. The worker owns their policy independently, meaning coverage persists even if the worker switches platforms. The insurer manages the risk pool, pricing engine, and payout execution directly.

3. Why This Model: Design Decisions

The problem statement explicitly requires AI risk assessment, dynamic weekly premiums, parametric triggers for weather and pollution, fraud detection, and instant payouts. These are not differentiators. Every competing team must deliver them. The five points below are design and research decisions that go beyond what is required. These differentiators are designed to improve real-world deployability, reduce basis risk, and ensure scalability across heterogeneous gig ecosystems.

Required vs. Differentiated Features

Required of ALL Teams	What We Do BEYOND Requirements
AI-powered dynamic premium calculation	D1: 7-platform cross-dataset AI training [1]-[7]
Parametric trigger automation	D2: Multi-level graduated severity bands [8][9]
Fraud detection mechanisms	D3: Manager-as-intelligence for social disruptions [10]
Weather, AQI, and curfew triggers	D4: Policy portability across all platforms [11]
Real-time monitoring and instant payout	D5: Composite disruption score (multi-signal fusion) [12]
Weekly pricing model	
Income loss coverage only	

D1: 7-Platform Cross-Dataset AI Trained Across the Full Q-Commerce Spectrum [1]-[7]

What It Is	Why No Other Team Will Have This
The AI models are trained on data derived from primary research across all seven Q-commerce platforms active in India: Swiggy Instamart, BigBasket Now, Flipkart Minutes, Amazon Now, FreshToHome Express, Zepto, and Blinkit [1]-[7]. Each platform has a distinct earnings structure, shift model, store type, and income volatility profile, all documented in seven dedicated ecosystem studies.	Most teams will build their risk model from one platform or from synthetic assumptions. A model trained on one platform misprices risk for all others. Zepto workers earn Rs. 50-55 per order on dense short routes; Blinkit workers earn Rs. 70 on pre-booked 5 km slots; Amazon Now workers earn Rs. 25-35 on longer routes [5][6][7]. Training across all seven produces a model accurate for the entire Q-commerce workforce. The dataset breadth is itself a research moat that cannot be replicated quickly.

All 7 Platforms in the Training Dataset

Platform	Store Model	Earn/Order	Monthly Income	Volatility Profile	Shift Type
Swiggy Instamart [1]	Dark stores	Rs. 50-75	Rs. 15K-19K	High volatility	Flexible login
BigBasket Now [2]	Warehouse/dark store	Rs. 35	Rs. ~20K	Stable	~13 hr shifts
Flipkart Minutes [3]	Micro-fulfil. hubs	Rs. 47	Rs. 14K-20K	Moderate	Flexible
Amazon Now [4]	Fulfillment centers	Rs. 25-35	Rs. 17K-22K	Low-moderate	Flexible gig
FreshToHome Express [5]	Food fulfil. centers	N/A	Rs. 17.5K-25.8K	Limited data	N/A
Zepto [6]	Dark stores	Rs. 50-55	Rs. 20K-21K	High density	Long cont. shifts
Blinkit [7]	Dark stores	Rs. ~70	Rs. 23K-29K	Slot-dependent	Pre-booked slots

D2: Multi-Level Graduated Severity Bands Payout Scaled to Actual Disruption Intensity [8][9]

What It Is	Why No Other Team Will Have This
The problem statement requires triggers but does not specify how triggers should fire. This system uses three severity levels per trigger, anchored to national standards. Rainfall (IMD): >= 64.5 mm -> Rs. 300, >= 115.6 mm -> Rs. 600, >= 204.5 mm -> Rs. 1,200. Temperature (IMD): >= 40 deg C -> Rs. 300, >= 45 deg C -> Rs. 600. AQI (CPCB): >= 300 -> Rs. 300, >= 400 -> Rs. 500 [8]. The payout a worker receives scales with the disruption they actually experienced.	Single-threshold parametric systems create basis risk, which is the core trust failure documented by PwC and the World Bank in parametric insurance adoption research [9]. A worker in 65 mm rain and one in 200 mm rain both receive the same amount under a flat trigger, which feels arbitrary and reduces willingness to renew. Graduated bands directly address this: the payout is proportionate and explainable, which is critical for adoption among low-income workers who are already skeptical of insurance products.

D3: Manager-as-Local-Intelligence Layer Human-Verified Detection for Social Disruptions [10]

What It Is	Why No Other Team Will Have This
For curfews, protests, and zone shutdowns, the system proposes a manager-driven detection	The problem statement requires curfew and strike detection but does not specify how. API-only

What It Is	Why No Other Team Will Have This
layer. Dark store managers flag disruptions via a dashboard, drawing on rider feedback, local awareness, and system signals (news APIs, government alerts). The system then checks route feasibility algorithmically: if no viable route exists to the delivery zone, payout triggers. If a route exists, no payout fires regardless of the flag. All manager inputs are logged and auditable, ensuring traceability and preventing misuse [10].	detection is noisy and delayed: news feeds report events after they have escalated, not as riders are being blocked. IMF research on social unrest forecasting confirms that ground-level human signals are faster and more locally accurate than any automated feed [10]. Route feasibility as the final arbiter also prevents both false positives and fraud.

D4: Policy Portability Across All 7 Platforms The Worker Owns Their Coverage [11]

What It Is	Why No Other Team Will Have This
The worker owns their individual policy independently of which platform they work on. Coverage remains active whether a worker is delivering for Zepto one week and Blinkit the next. The platform acts only as a distribution partner: it never owns, pays for, or controls the policy. Premium is deducted from weekly earnings via the active platform but the policy belongs to the worker [11].	Existing gig worker insurance (e.g., Zepto's GPA group policy) is platform-owned. If a worker switches platforms or goes inactive, coverage disappears instantly [11]. Given that 33% of gig workers who quit a platform eventually return, often using it as emergency income, many workers rotate across platforms regularly. Policy portability is the only design that provides continuous protection for a workforce that is structurally multi-platform. No current product offers this.

D5: Composite Disruption Score Multi-Signal Fusion Beyond Single-API Triggers [12]

What It Is	Why No Other Team Will Have This
Rather than firing payouts on a single API signal, the system uses a weighted Composite Disruption Score combining rainfall, AQI, traffic congestion index, and zone flood alert: Composite Score = w1 x R_norm + w2 x AQI_norm + w3 x Traffic_norm + w4 x Flood_flag. All inputs are normalized to a common scale to ensure comparability across signals. Trigger bands are evaluated against this composite, not against any one variable in isolation. Weights are tuned per city based on historical disruption patterns [12].	The problem statement says to use weather APIs and traffic data, but using them as independent, parallel triggers means a heavy-rain day with moderate AQI and high congestion may not cross any single threshold, even though the combined effect has halted deliveries. A composite score captures the real-world truth: disruptions are multi-dimensional. This design also makes the system more robust against API failures: if one data source is unavailable, the score degrades gracefully rather than failing to trigger at all [12].

Differentiator Summary

#	Differentiator	What It Replaces	Evidence Source
D1	7-platform cross-dataset	Single-platform or synthetic data	Ecosystem Studies [1]-[7]
D2	Multi-level graduated severity bands	Single threshold: flat or no payout	IMD / CPCB standards [8][9]
D3	Manager-as-intelligence layer	API-only detection of social disruptions	Curfews and Protests doc [10]
D4	Policy portability across platforms	Platform-owned GPA group policy (non-portable)	PARAMETERS.docx [11]
D5	Composite disruption score	Single-signal triggers (weather OR AQI OR traffic)	GPS and Data Flow doc [12]

4. End to End System Flow

The system is structured as a six-stage pipeline. Each stage has clearly defined inputs, a processing function, outputs, and data sources. Stages 1 to 4 operate at policy inception (weekly), while Stages 5 and 6 operate continuously throughout the policy period.

Each step is represented as a self-contained visual block showing what happens, why it exists, and the design justification behind it.

Step 1 — Worker Registration

What Happens: The worker registers by providing profile and operational data, including platform, city, vehicle type, shift hours, experience level, and delivery zone.

Why It Exists: The system requires worker-specific data to determine exposure to environmental and operational risks. Without this information, risk classification and eligibility for payouts cannot be established.

Design Justification: Gig worker income varies significantly based on location and working patterns. Associating each worker with a geo-zone enables region-specific risk modeling, ensuring that pricing and trigger evaluation reflect actual conditions.

▼

Step 2 — Risk Calculation

What Happens: The system computes risk_score, claim_probability, and expected_loss using environmental exposure, disruption probabilities, and income patterns. Expected income is derived from delivery volume and earnings per order.

Why It Exists: Insurance pricing must be based on expected future loss. Without quantifying risk, premiums would not reflect actual exposure, leading to financial imbalance.

Design Justification: Disruption probabilities and average loss values are used to estimate expected weekly loss. This converts raw worker data into a measurable financial risk, forming the foundation for actuarial pricing.

▼

Step 3 — Weekly Premium Generation

What Happens: The system converts expected loss into a weekly premium by adding expense loading and a risk margin. A tiered pricing model is offered with corresponding payout levels.

Why It Exists: Premium generation translates calculated risk into a usable financial product. It ensures that expected payouts, operational costs, and uncertainty are accounted for.

Design Justification: A weekly model aligns with gig workers' income cycles. Tiered pricing balances affordability and coverage, while partial payouts reduce moral hazard and maintain sustainability.

▼

Step 4 — Policy Subscription

What Happens: The worker selects a plan and activates a 7-day policy with predefined rules, including payout limits and maximum events per week.

Why It Exists: A policy establishes contractual boundaries under which monitoring and payouts operate. Without defined terms, trigger events cannot be evaluated consistently.

Design Justification: Event caps and payout limits control insurer exposure. Fixed rules ensure predictability, which is essential for parametric insurance systems.

▼

Step 5 — Real-Time Monitoring

What Happens: The system continuously collects real-time data such as rainfall, temperature, AQI, flood alerts, and platform status through external APIs.

Why It Exists: Parametric insurance depends on objective and continuously updated data. Monitoring enables immediate detection of disruptions without relying on user input.

Design Justification: Independent data sources ensure transparency and prevent manipulation. Continuous monitoring guarantees that events are captured regardless of when they occur.

▼

Step 6 — Event Occurrence

What Happens: A disruption event occurs within the worker's zone, affecting delivery activity and reducing income. The system records the event and associated parameters.

Why It Exists: The insurance model is triggered by disruptions that directly impact income. Identifying these events establishes the basis for payout eligibility.

Design Justification: Separating event detection from payout logic ensures objectivity. Income loss is a consequence of disruption, but the system focuses on measurable triggers rather than subjective reporting.

▼

Step 7 — Trigger Activation

What Happens: The system evaluates whether predefined thresholds are crossed. If conditions are satisfied and the worker is active, the trigger is activated.

Why It Exists: Trigger-based activation replaces traditional claim assessment, enabling automated and deterministic decision-making.

Design Justification: Thresholds are based on standardized classifications such as rainfall levels and AQI categories. Multi-level trigger bands align payout with event severity, reducing mismatch between disruption intensity and compensation.

▼

Step 8 — Payout Execution

What Happens: Once triggered, the system automatically transfers a predefined payout to the worker. No claim submission or verification is required.

Why It Exists: Gig workers require immediate financial support. Delayed claim processing would reduce the effectiveness of income protection.

Design Justification: Fixed payouts eliminate administrative overhead and enable rapid disbursement. The system provides partial income replacement, ensuring liquidity while maintaining financial sustainability.

Example Scenario: Swiggy Instamart Delivery Partner Mumbai Monsoon Rainfall Event [1][8]

Worker	Swiggy Instamart Partner: Mumbai (High Rainfall Zone) [1]
Plan Subscribed	Standard Plan: Rs. 79/week, Rs. 400 payout/event
Policy Status	Active: Day 3 of 7 | Events Used: 0 of 2
Normal Daily Income	35 orders x Rs. 43/order = Rs. 1,500
Disruption Event	Monsoon rainfall: 128 mm recorded by Weather API
IMD Classification [8]	Very Heavy Rain: Level 2 threshold (>= 115.6 mm)
Disrupted Daily Income	15 orders x Rs. 40/order = Rs. 600
Income Loss	Rs. 1,500 - Rs. 600 = Rs. 900 (60% reduction)

Trigger Evaluation: Observed rainfall of 128 mm is evaluated against IMD-aligned thresholds.
Level 1 (>= 64.5 mm): exceeded.
Level 2 (>= 115.6 mm): exceeded.
Level 3 (>= 204.5 mm): not met.
Highest activated level: Level 2.
Worker active in registered disruption zone: confirmed.
Trigger Event = TRUE.
Standard Plan Level 2 payout of Rs. 600 is automatically issued. [8]

Payout Outcome

Metric	Value
Delivery Earnings (Disruption Day)	Rs. 600
Insurance Payout (Level 2 Trigger)	+ Rs. 600
Total Income on Disruption Day	Rs. 1,200
Expected Income (Normal Day)	Rs. 1,500
Residual Basis Risk	Rs. 300 (by design: partial coverage, not full indemnity)
Income Recovery Rate	67% of loss recovered

Metric	Value
Events Remaining This Week	1 of 2

The Rs. 600 payout reflects Level 2 severity, not Level 1, because 128 mm exceeds the Very Heavy Rain threshold of 115.6 mm [8]. A single-threshold system would have paid only Rs. 300, illustrating precisely why tiered bands are essential.

The residual basis risk of Rs. 300 is intentional: the system provides liquidity support, not full indemnification. The 2-event weekly cap and Rs. 79 premium together maintain actuarial sustainability.

5. Parametric Trigger System

The parametric trigger system forms the core decision engine of ***BHIMA ASTRA***. Rather than relying on subjective loss assessment, it uses objective, pre-defined thresholds sourced from India Meteorological Department (IMD) and Central Pollution Control Board (CPCB) national standards [8]. When real-time data crosses a threshold and the worker is confirmed active in the affected zone, the payout fires automatically.

Trigger Thresholds and Payout Levels [8][9]

Trigger Type	Level	Threshold	Standard	Basic Plan	Standard Plan	Premium Plan
Heavy Rainfall [8]	L1	>= 64.5 mm/day	IMD Heavy Rain	Rs. 300	Rs. 400	Rs. 600
Heavy Rainfall [8]	L2	>= 115.6 mm/day	IMD Very Heavy Rain	Rs. 300	Rs. 600	Rs. 900
Heavy Rainfall [8]	L3	>= 204.5 mm/day	IMD Extremely Heavy Rain	Rs. 600	Rs. 800	Rs. 1,200
Extreme Heat [8]	L1	>= 40 deg C	IMD Heatwave	Rs. 300	Rs. 300	Rs. 300
Extreme Heat [8]	L2	>= 45 deg C	IMD Severe Heatwave	Rs. 300	Rs. 600	Rs. 600
Air Pollution [9]	L1	AQI >= 300	CPCB Very Poor	Rs. 300	Rs. 300	Rs. 300
Air Pollution [9]	L2	AQI >= 400	CPCB Severe	Rs. 300	Rs. 500	Rs. 500
Flood / Zone Shutdown [8]	--	Flood alert = 1	State / IMD alert	Plan payout	Plan payout	Plan payout
Platform Outage / Curfew	--	Outage flag or manager-verified	System/Manager	Plan payout	Plan payout	Plan payout

Composite Disruption Score [12]

Rather than evaluating triggers independently, the system computes a weighted Composite Disruption Score that fuses multiple environmental signals. This ensures that combined adverse conditions, even when no single variable crosses its threshold, are properly captured and evaluated [12].

Composite Score = w_1 × R_norm + w_2 × AQI_norm + w_3 × Traffic_norm + w_4 × Flood_flag

Where:

- w_1, w_2, w_3, w_4 are weights tuned per city based on historical disruption patterns [12]
- R_norm = Normalized rainfall
- AQI_norm = Normalized air quality index
- Traffic_norm = Normalized traffic congestion level
- Flood_flag = Binary indicator (0 or 1) for flood presence

All inputs are normalized to a common scale (0 to 1) to ensure comparability across signals.

This design makes the system more robust against API failures. If one data source is unavailable, the composite score degrades gracefully rather than failing to trigger at all [12].

6. AI and Machine Learning Architecture

The AI layer is designed to enable data-driven, actuarially consistent, and personalised insurance pricing for gig delivery workers operating under uncertain environmental conditions. The system integrates three interconnected models: an Income Prediction Model, a Disruption Risk Prediction Model, and a Premium Adjustment Model.

These models operate as a closed actuarial decision loop, where each stage directly feeds into the next:

$$E[L_{(weekly)}] = P_{(disruption)} \times (\hat{Y}_{(income)} \times S_{(event)})$$

$\hat{Y}_{(income)}$ = predicted weekly income of the worker

$P_{(disruption)}$ = probability of a disruption event in the worker's zone

$S_{(event)}$ = severity factor from parametric trigger thresholds [Section 5]

6.1 Income Prediction Model

Purpose

The Income Prediction Model estimates the expected daily and weekly earnings of a gig delivery worker under normal, undisrupted operating conditions. This predicted income serves as the baseline for all downstream insurance computations, including income loss estimation, coverage determination, and premium pricing.

Unlike salaried employment, gig worker income is inherently stochastic and depends on multiple interacting factors such as order volume, working hours, surge demand, incentives, and geographic demand variability [2][3][4]. Empirical analysis across multiple quick-commerce platforms shows that daily earnings range from Rs. 560 to Rs. 3,000, with an average of approximately Rs. 1,350, while weekly earnings vary between Rs. 4,000 and Rs. 21,000 depending on worker activity and zone characteristics [1][5].

Algorithm: Random Forest Regressor [13]

The income prediction task is formulated as a supervised regression problem and implemented using a Random Forest Regressor. This model is selected due to its strong performance on structured, high-dimensional datasets and its ability to capture complex, non-linear relationships between multiple input variables [13][14].

Random Forest operates by aggregating predictions from multiple decision trees trained on different subsets of the data, resulting in reduced variance and improved generalisation, robustness to noise and outliers (e.g., surge spikes), and the ability to model interaction effects without explicit feature engineering. Studies in income and salary prediction consistently demonstrate that ensemble tree-based models outperform linear regression in multi-factor scenarios involving behavioural and contextual features [14][15].

Model Inputs

Variable	Description	Type	Source
Orders per Day	Number of deliveries completed	Observed	[2][3][4]
Earnings per Order	Pay per delivery	Observed	[2][3][4]
Shift Hours	Active working hours per day	Observed	[2]
Surge Multiplier	Demand-based pricing multiplier	Observed	[2][3]
Geo-Zone ID	Worker's delivery zone	Observed	[5]
Zone Demand Level	Low / Medium / High demand	Derived	[5]
Day of Week	Demand variation pattern	Derived	[5]
Experience Level	Worker tenure (years)	Observed	[5]

Model Outputs

Output Variable	Description	Range	Role in System
expected_income	Predicted daily earnings	Rs. 560 to Rs. 3,000	Baseline income estimation
income_baseline_weekly	Predicted weekly earnings	Rs. 4,000 to Rs. 21,000	Coverage and pricing anchor

Income Loss Estimation

$$\text{Loss} = \hat{Y}_{(income)} \times S_{(event)}$$

$S_{(event)}$ = severity of disruption derived from parametric triggers (e.g., rainfall thresholds, AQI levels)

Empirical evidence shows disruption events reduce earnings by 30-60% [2][3][4]

Example: Baseline income = Rs. 1,500 / day

After heavy rainfall = Rs. 600 (income drops approx. 60%)

Loss = Rs. 900

6.2 Disruption Risk Prediction Model

Purpose

The Disruption Risk Prediction Model estimates the probability that one or more parametric trigger events will occur in a worker's delivery zone during the upcoming 7-day policy period. Unlike traditional insurance models that rely on static, city-level averages, this model generates zone-specific, time-sensitive probability estimates, allowing the system to dynamically adjust risk based on real environmental and operational conditions [16][17].

Algorithm: XGBoost Classifier [16]

The disruption prediction problem is formulated as a supervised classification task and implemented using XGBoost (Extreme Gradient Boosting). XGBoost is selected due to its strong performance on structured datasets, particularly in scenarios involving environmental variables and risk prediction [16][17].

Model Inputs

Variable	Description	Range	Type	Source
Rainfall (mm/day)	Daily precipitation level	0-300 mm	Observed	[8]
Temperature (deg C)	Ambient outdoor temperature	20-48 deg C	Observed	[8]
AQI	Air Quality Index	0-500	Observed	[9]
Flood Alert Flag	Flood occurrence indicator	0 / 1	Observed	[8]
Platform Outage Flag	App/service disruption	0 / 1	Observed	[1]
Zone Shutdown Flag	Restricted access indicator	0 / 1	Observed	[1]
Traffic Index	Road congestion level	0-100	Observed	[18]
Historical Disruption Frequency	Past event rate in zone	Derived	Derived	[5]

Model Outputs

Output Variable	Description	Range	Role in System
P(Rain Event)	Probability of rainfall trigger	0.15-0.25	Risk estimation
P(Heat Event)	Probability of heatwave trigger	0.10-0.20	Risk estimation
P(AQI Event)	Probability of pollution trigger	0.05-0.15	Risk estimation
combined_disruption_probability	Overall weekly disruption probability	0.30-0.40	Core premium driver
zone_risk_score	Normalised risk score for zone	0.0-1.0	Pricing adjustment

$$\boldsymbol{E[L_{weekly}] = P_{disruption} \times (\widehat{Y}_{income} \times S_{event})}$$

Using representative values: [6][7]

1. $P_{disruption} = 0.35$
2. Expected loss per event = ₹600

$$E[L_{weekly}] = 0.35 \times ₹600 = ₹210$$

6.3 Premium Adjustment Model (Dynamic Pricing) [19][20]

Purpose

The Premium Adjustment Model converts the expected loss derived from Models 6.1 and 6.2 into a final, personalised weekly premium for each worker. This model ensures that pricing is not fixed or uniform but dynamically reflects the worker's income level, geographic exposure, and environmental risk conditions. Unlike traditional insurance systems where all users in a category pay the same premium, this system introduces intra-plan personalisation, improving actuarial fairness and reducing adverse selection [19][20].

$$\boldsymbol{Weekly\ Premium = E[L] + Expense\ Loading + Risk\ Margin}$$

Where:

1. $E[L]$ = Expected weekly loss
2. Expense Loading = 10–15% (operational costs)
3. Risk Margin = 20–30% (uncertainty buffer)

Using representative values:

$$Weekly\ Premium = ₹210 + ₹21 + ₹52 = ₹283$$

This represents the **full-risk premium**, i.e., the theoretical price required to fully cover expected losses and operational costs [6].

Tiered Plan Structure

Plan	Weekly Premium	Payout per Event	Max Events/Week	Max Payout	Target Segment
Basic	Rs. 49	Rs. 300	2	Rs. 600	Low-income / part-time
Standard	Rs. 79	Rs. 400	2	Rs. 800	Regular workers
Premium	Rs. 119	Rs. 600	2	Rs. 1,200	High-income / full-time

Dynamic Adjustment Factors

Factor	Effect on Premium	Source
Geographic Zone Risk	High-risk zones increase premium within plan band	[5][16]
Seasonal Risk	Monsoon / extreme weather periods increase premium	[8]
Historical Disruption Freq.	Frequent past events increase premium	[5]
Worker Income Exposure	Higher earnings implies higher insurable loss implies higher premium	[5]
Tenure and Consistency	Stable workers receive discount within plan band	[1][19]

Dynamic pricing in insurance has been shown to improve risk alignment and reduce adverse selection by ensuring that lower-risk users are not overcharged while higher-risk users are priced appropriately [19][20][21].

Integration with Parametric System

Once the premium is finalised and the worker subscribes, the system monitors environmental triggers in real time (Section 5). When a predefined threshold is crossed, payout is executed automatically. No claim submission or verification is required. The AI layer therefore determines how much to charge while the parametric system determines when to pay. This separation ensures

that the system remains fully parametric, automated, and scalable, while still leveraging AI for accurate risk-based pricing.

7. Adversarial Defense and Anti-Spoofing Strategy

Parametric insurance fires automatic payouts on objectively measured events like rainfall, AQI, heatwave, and flood alert, thus removing all administrative friction. This same design, however, changes the fraud surface entirely: because payouts trigger on environmental data rather than individual declarations, the attack vector shifts from falsifying loss documentation to falsifying presence in the disrupted zone at the moment of the trigger.

In Q-commerce gig insurance this takes two principal forms:

(1) Individual GPS spoofing, where a worker overrides the phone's GPS output using a freely available Android app to report disruption-zone coordinates while remaining at home.

(2) Coordinated ring fraud, where a Telegram-organised syndicate of workers simultaneously spoof their location, flooding the claims pipeline in a narrow window and draining the liquidity pool before any rule-based check responds.

The DEVTrails 2026 threat report confirmed this precisely: 500 workers exploited a beta platform through coordinated GPS spoofing, triggering mass false payouts in under fifteen minutes. Preventing this attack class is the primary design objective of the fraud detection system.

Core Design Principle
BHIMA ASTRA treats fraud detection as a signal separation problem:
distinguishing real-world disruption behavior from simulated presence under adversarial conditions.

7.1 Limitations of Existing Approaches

Rule-based systems are the most widely deployed fraud control in insurance [22], but they have no temporal memory and evaluate each claim at a single instant, unable to distinguish a genuine worker who lost GPS lock in a storm from one spoofing coordinates at home. Isolation Forest detects statistical outliers [23] but scores every claimant in isolation: a ring of 200 workers filing simultaneously appears as 200 individually marginal anomalies, because the algorithm cannot represent the connections between them [24]. XGBoost excels on tabular claim data [16][25] but processes each record as an independent row and cannot learn that a worker stationary for 40 minutes before a trigger, who files within 60 seconds of its timestamp, presents a fundamentally different behavioral profile from an active worker.

The Key Distinction
Traditional models ask: "Is this unusual?"
BHIMA ASTRA asks: "Is this physically plausible and socially independent?"
The distinction is the entire design.

7.2 Original Contributions

Contribution	Description
C1: Cost-cascade architecture	~80% of claims resolved at the rule layer with zero ML overhead; expensive models invoked only at the 4-20% margin.
C2: Temporal and relational hybrid	LSTM behavioral sequence model (time axis) combined with graph community detection (relational axis). No prior parametric insurance fraud system combines both.
C3: Agent-based orchestration	Adaptive cluster-context escalation, stateful 48-hour hold/release lifecycle, and weekly online learning from confirmed fraud outcomes.
C4: Parametric-specific design	Covers all five Q-commerce disruption triggers: rainfall, AQI, heatwave, flood alert, and platform outage. Designed for gig workers, not generic insurance claimants.

7.3 Core Detection Architecture

Stage 1: Deterministic Filtering Layer (~80% of claims)

Resolves approximately 80% of claims with zero ML overhead through contract rules and four anti-spoofing checks:

- GPS and cell tower delta: Spoofing apps alter GPS output but cannot change tower registration. Delta > 500 m flags the claim [23].
- Accelerometer and motion mismatch: A flat accelerometer reading for > 10 min during a trigger window is a static-presence signal.
- Timing implausibility: A claim filed within 60 seconds of a trigger timestamp is physically implausible for a genuine notification-read-interact workflow.
- Device fingerprinting: A previously flagged device ID triggers an immediate audit hold. Clean claims score 0.0 and are paid within 5 minutes.

Stage 2: Behavioral Intelligence Layer (~15% of claims)

A lightweight LSTM network evaluates the 10-minute behavioral time series around the trigger event [26]. The fraud signal lies in the sequence and rhythm of features, not any single snapshot. A genuine rider shows GPS jitter correlated with motion spikes, app interactions, and tower transitions; a spoofer at home shows a flat accelerometer, zero app events, and a single static tower. No non-sequential model captures this distinction [26][22].

Architecture: 2 LSTM layers, 32 hidden units, 10 time steps x 5 features, ~25,000 parameters, less than 5 ms CPU inference [26][27].

Score Range	Action
Score < 0.3	Full payout released instantly
Score 0.3-0.7	50% released immediately, 50% held 48 h (auto-released if no escalation)
Score > 0.7	Full hold, placed in audit queue

Feature	Description	Range	Type
GPS and cell tower delta	GPS vs. triangulated cell tower position	0-10,000 m	Continuous
Accelerometer variance	Movement intensity across x/y/z axes	0.0-50.0	Continuous
App interaction count	User-initiated events per minute	0-60	Integer
Cell tower transition	New tower connection within minute	0 / 1	Binary
Order status event	Delivery status change within minute	0 / 1	Binary

Stage 3: Relational Intelligence Layer (~4% of claims)

Individual LSTM scores for ring members typically sit at 0.25-0.30, each plausible alone. Yet the ring leaves an unmistakable relational signature: same filing window, GPS clustered within an impossible radius, identical device model and OS, same IP subnet [28][29][24]. For each trigger event, a claim graph is constructed where nodes represent workers and edges connect those sharing multiple signals such as time window, location proximity, device similarity, or IP subnet. Louvain community detection [29] identifies dense clusters of coordinated workers; groups above threshold size are flagged as potential fraud rings and the entire cohort is held. Phase 3 upgrades to a GCN (PyTorch Geometric) with learned edge weights [30][31].

Stage 4: Decision Intelligence Layer (~1% of claims)

The Fraud Agent acts as the system's decision layer, combining signals across stages and enabling context-aware escalation that no standalone model can achieve. A worker scoring 0.28, individually below threshold, who belongs to a cluster of 14 workers all scoring 0.22 to 0.30 is routed to Stage 3, because the agent holds the joint cluster distribution in context. It manages the full claim lifecycle: escalation, 48-hour hold decisions, and automated release, ensuring consistent and stateful execution. Confirmed outcomes feed the Insight Agent's weekly retraining cycle, continuously improving LSTM and GNN accuracy from real operational data, a feedback loop absent from every static fraud pipeline.

7.4 Cost Cascade and System Advantages

Stage	Model	Claims %	Cost	Fraud Signal Detected
Stage 1	Rule engine	~80%	~0 ms	Contract breach, GPS-tower delta, timing, device

Stage	Model	Claims %	Cost	Fraud Signal Detected
Stage 2	Tiny LSTM	~15%	~5 ms CPU	Individual behavioral sequence anomalies
Stage 3	Louvain / GCN	~4%	~50 ms CPU	Coordinated ring / syndicate fraud
Stage 4	LLM Agent (RAG)	~1%	~500 ms API	Edge cases and full audit documentation

7.5 Fraud Resilience Scenarios

Scenario	Situation	System Response	Result
Mass GPS Spoofing Attack	500 workers simultaneously activate GPS spoofing apps and file claims during a Tier 2 rainfall trigger in Mumbai.	Stage 3 constructs a claim graph; Louvain detects a >8-node community with high shared-attribute edge weights.	Entire ring blocked before payout. Fraud contained within 90 seconds.
Genuine Worker: Network Failure During Disruption	Worker in a flood-affected zone loses mobile connectivity for 18 minutes, producing erratic GPS patterns that resemble spoofing.	Score 0.41: 50% payout released immediately. Stage 4 LLM agent cross-references the Monitor Agent's network outage log. Outage confirmed.	Remaining 50% released within 4 hours. Worker receives full payout. No manual intervention required.
Coordinated Low-Score Ring	18 workers each score 0.24-0.29 on the LSTM, individually below the 0.30 escalation threshold.	Fraud Agent observes the cluster joint distribution and routes the full cohort to Stage 3. Louvain confirms a dense community.	Entire cohort held. Cluster-context escalation exposes coordinated fraud that individual-level models would completely miss.

8. Multi-Agent System Design

The system employs a multi-agent architecture that coordinates real-time monitoring, fraud detection, claims processing, and decision escalation across specialized agent roles. Each agent is purpose-built for a specific function within the insurance pipeline, enabling parallel processing, stateful lifecycle management, and continuous improvement through online learning.

Agent	Role	Key Capability
Monitor Agent	Continuously ingests real-time API feeds (weather, AQI, traffic, platform status)	Triggers composite disruption score evaluation per zone
Trigger Agent	Evaluates IMD/CPCB thresholds and composite score against active policies	Fires trigger events and determines severity level (L1/L2/L3)
Fraud Agent	Orchestrates the 4-stage fraud detection cascade	Stateful 48-hour hold/release lifecycle; cluster-context escalation
Payout Agent	Executes approved payouts via UPI/Razorpay sandbox	Ensures immediate transfer for clean claims; updates event counter
Manager Intelligence Agent	Processes dark store manager flags for social disruptions (curfews, protests)	Route feasibility check as final arbiter for social trigger payouts [10]
Insight Agent	Runs weekly retraining cycle on confirmed fraud outcomes	Online learning from operational data; continuously improves LSTM and GNN accuracy

The D3 differentiator, the **Manager-as-Local-Intelligence Layer**, is embedded within the Manager Intelligence Agent [10]. Dark store managers flag disruptions via a dashboard, drawing on rider feedback, local awareness, and system signals. The system checks route feasibility algorithmically: if no viable route exists to the delivery zone, payout triggers. **All manager inputs are logged and auditable, ensuring traceability and preventing misuse.**

9. Financial Model

THE MARKET OPPORTUNITY

23.5 M Gig workers in India by 2030	**₹64,000 Cr** Q-commerce India market size 2025–26	**90%** Gig workers with zero savings today
20–30% Monthly income lost during disruptions	**3 M+** Delivery partners on Blinkit/Zepto/Instamart	**No structured income protection** Existing income protection for gig workers

These six numbers define the gap this system fills. Millions of workers, zero protection, a measurable and insurable risk.

9.1 Delivery Partners (The Worker)

Their Reality Today

Metric	Value
Net monthly income (full-time)	~Rs. 18,763 after fuel and maintenance costs [32]
Operating expenses	~32% of gross earnings consumed [32]
Workers struggling to cover basics	35% cannot cover food and rent regularly [32]
Income drop on a disrupted day	Rs. 560-900 lost per event day [7][10]
Existing income protection	None (structured) [6]

What *BHIMA ASTRA* Delivers for Workers

Benefit	Detail
Automatic payout on disruption: no claim filing	Rs. 300-1,200 transferred automatically with minimal delay after trigger validation. [10][11]
Affordable: just 1.0% - 2.5% of weekly income	Rs. 49-119/week premium on a ~Rs. 4,690/week income. Less than Rs. 17/day at max. [32][10]
Platform-enabled enrolment with opt-out	No separate app or form. Worker is protected from day one. Proven in Uber and fintech insurance. [33]
Standard plan Rs. 800 payout: 17% income replacement	On a disrupted week where income falls to Rs. 600, Rs. 800 is meaningful liquidity. [32][10]

Benefit	Detail
Evidence from real-world pilot implementations	Jan Sahas parametric pilot (India): informal workers received Rs. 1,000 payouts during heatwaves. Same trigger model. [34]

9.2 Delivery Platforms

Why Platforms Will Participate

Value Proposition	Detail
Worker retention: the #1 platform pain point	33% of gig workers who quit eventually return. Constant re-onboarding is expensive. Welfare benefits are the top retention driver. [32][35]
Regulatory compliance at zero premium cost	Rajasthan Gig Workers Act 2023 and Karnataka Bill 2024 mandate worker welfare contributions. Platform distributes but never underwrites. [36][37]
Distribution commission: 5-10% per policy	10,000 policies x Rs. 75 avg x 7.5% = Rs. 56,250/week earned by platform. No insurance liability at all. [33]
Public reputation: a verifiable welfare story	Swiggy introduced accident cover due to media and protest pressure. [38] Our model gives platforms the same story at zero premium cost.
No financial liability	Platform is distributor only. It never underwrites risk, never pays claims, and never owns any policy.

Platform Commission: Structured Illustration

Scenario: 1 platform, 1 major city (e.g. Blinkit, Mumbai)
Active delivery fleet = 50,000 workers
Adoption rate (conservative 20%) = 10,000 policies
Average weekly premium = ₹75
Platform commission rate = 7.5%

→ **Weekly commission income = ₹56,250**
→ **Annual commission income = ₹29.2 Lakh (single city, 1 platform)**

Note: Commission earned purely from distribution — no capital, no risk.

9.3 Insurance Provider (The Insurer)

Structural Advantage	Detail
Near-zero customer acquisition cost	Platform brings pre-qualified policyholders directly. No agent network, no ads, no per-customer marketing spend. [33]
52 premium payments per year per worker	Weekly model = 52 revenue events/year vs. 1 for annual insurance. Stable, predictable, high-frequency premium inflow. [10]
Zero claim investigation cost	Parametric model: trigger fires implies payout transfers automatically. No adjuster, no loss assessment, no dispute. [11]
AI risk pricing reduces adverse selection	XGBoost zone_risk_score prices each worker by actual zone and season. High-risk zones priced higher. [10]
Risk pool cross-subsidy	Premium plan (Rs. 119) cross-subsidises Basic plan (Rs. 49). Larger pool = more stable loss ratio over time. [10]

9.4 Revenue Model: Phased Rollout

Phase 1: Pilot Deployment (1 platform x 1 city)

Parameter	Value
Active delivery workforce	30,000 workers
Adoption rate (conservative)	10%
Policyholders	3,000 workers
Average weekly premium	Rs. 70
Annual premium inflow	3,000 x Rs. 70 x 52 = approx. Rs. 1.09 Crore
Annual claims estimate	3,000 x 0.55 x 9 x Rs. 400 = approx. Rs. 59.4 Lakh
Loss ratio	~54% (within target actuarial range of 50-65%)
Surplus	Rs. 1.09 Cr - Rs. 0.70 Cr = Rs. 0.39 Crore
Break-even threshold	~2,000-2,500 policyholders

Phase 2: Controlled Expansion (2-3 cities x 2 platforms)

Parameter	Value
Worker pool	~150,000
Adoption rate	15%
Policyholders	~22,500

Parameter	Value
Average weekly premium	Rs. 75
Annual premium	approx. Rs. 8.8 Crore
Annual claims	22,500 x 0.55 x 9 x Rs. 400 = approx. Rs. 4.45 Crore
Loss ratio	~50-55%

Phase 3: Mature Scale (3 platforms x major metro clusters)

Parameter	Value
Worker pool	~500,000
Adoption rate	20%
Policyholders	100,000
Average weekly premium	Rs. 75
Annual premium	approx. Rs. 39 Crore
Annual claims	100,000 x 0.55 x 9 x Rs. 400 = approx. Rs. 19.8 Crore
Loss ratio	~50-60%

9.5 Sensitivity Analysis

Scenario	Parameter Change	Impact	Result
High Disruption Year	Disruption weeks: 9 to 12	Claims up 33%	Claims approx. Rs. 26 Cr (Phase 3); Loss ratio approx. 65-68%; still within acceptable actuarial range
Low Adoption	Adoption: 20% to 10%; Policyholders: 100K to 50K	Premium and claims both halve	Loss ratio remains stable; model scalable but slower to profitability
Severe Event Spike	Payout/event: Rs. 400 to Rs. 600	Claims increase ~50%	Loss ratio may reach ~70%; mitigated through payout caps, tiered coverage, risk-based pricing

Key Insight

The model is robust under variability, with loss ratios remaining within acceptable bounds across multiple stress conditions.

10. Stakeholder Benefits

Stakeholder	Primary Value	Key Number
Delivery Partner (Worker)	Automatic income payout on disruption day with no claim needed	Rs. 300-1,200 payout; 1.0-2.5% of weekly income [10][11]
Delivery Platform	Worker retention, regulatory compliance, and commission income	5-10% commission; zero premium liability [33][36][37]
Insurance Provider	High-frequency weekly revenue; low-cost parametric operations	Rs. 39 Cr/yr revenue; loss ratio improves with scale [10]

This combination of scale, data availability, and measurable disruption risk makes gig worker income protection uniquely suitable for parametric insurance design.

11. Advantages of Our Model

Advantage	Description
Fully Automated Payouts	No claim filing, no adjuster, no delay. Payouts execute automatically within minutes of a confirmed trigger.
AI-Personalised Pricing	Premiums reflect each worker's actual risk profile, not a flat rate. This improves fairness and reduces adverse selection.
Multi-Level Trigger Bands	Payouts scale with disruption severity, reducing basis risk and increasing worker trust in the product.
Platform Distribution Model	Zero customer acquisition cost for the insurer. Platforms distribute the product and earn commission without taking on insurance liability.
Policy Portability	Workers retain coverage regardless of which platform they work on: the only design that matches the multi-platform reality of gig work.
Composite Disruption Score	Captures combined adverse conditions that no single-signal trigger would detect, improving trigger accuracy and reducing false negatives.
Fraud-Resilient Architecture	4-stage cost-cascade system resolves ~80% of claims instantly, while protecting the liquidity pool from coordinated ring attacks.
Actuarially Sustainable	Loss ratios projected at 50-65% across all phases, with break-even achieved at ~2,000-2,500 policyholders in pilot deployment.
Weekly Renewal Cycle	Aligns with gig workers' income rhythm, enabling dynamic weekly repricing and 52 premium revenue events per year.
Regulatory Alignment	Designed to help platforms meet obligations under the Rajasthan Gig Workers Act (2023) and Karnataka Gig Workers Bill (2024) [36][37].

12. Limitations and Future Improvements

Known Limitations

Limitation	Description	Mitigation in Current Design
Basis Risk	Mismatch between actual income loss and fixed parametric payout. A worker's actual loss may differ from the predefined payout.	Mitigated by multi-level severity bands (D2), but not fully eliminated. By design: full indemnification is not the model's goal.
Data Dependency	Model accuracy depends on platform data availability. In early deployment, historical training data may be limited or inconsistent across platforms.	System designed to function with self-reported data as fallback. 7-platform cross-dataset reduces single-source dependence.
GPS Spoofing Evolution	Adversarial actors may develop more sophisticated spoofing methods that challenge the deterministic fraud layer.	Stage 2 LSTM and Stage 3 graph-based detection provide adaptive layers; weekly online retraining allows the system to evolve with attacker strategies.
Coverage Exclusions	Health, life, accident, and vehicle repair claims are explicitly out of scope. Workers face additional vulnerabilities beyond income disruption.	Scope kept narrow to ensure actuarial sustainability. Expansion considered for Phase 3 and beyond.
Urban Concentration	Initial deployment targets Tier-1 cities where API data coverage is reliable. Tier-2 and rural coverage may be limited.	Phased rollout design accounts for this; Tier-2 expansion planned in Phase 2.

Future Improvements (Planned for Phase 2 and Beyond)

- Seasonal model recalibration: Automated re-training of AI models at the onset of monsoon, summer, and winter seasons to maintain prediction accuracy.
- Expanded platform coverage: Onboarding of additional Q-commerce and hyperlocal delivery platforms beyond the current seven.
- Real-time claim audit trail: Immutable, time-stamped audit logs accessible to both insurer and worker for full transparency.
- Tier-2 city expansion: API infrastructure and manager intelligence layer adaptation for smaller urban centres.
- Health and accident micro-coverage: Exploratory integration of basic accident cover as an add-on to the income protection plan.
- GCN upgrade (Phase 3): Transition from Louvain community detection to PyTorch Geometric GCN for more sophisticated ring fraud detection with learned edge weights.

- Worker dashboard: A mobile-native earnings and coverage view, enabling workers to track policy status, trigger events, and payout history in real time.

13. Technology Stack

The technology stack is designed for real-time data ingestion, low-latency inference, and scalable payout execution. Full stack details will be provided in Phase 2 submission.

> GigShield is a seven-layer system where each layer has a specific job, uses the right tool for that job, and connects to adjacent layers through well-defined contracts. The MVP is computationally lean (cascade inference, CPU-first ML), resilient (stateful agent orchestration), and independently testable. Each layer can be mocked, swapped, or scaled without touching the others.

LAYER 1 | Data Ingestion Layer
Environmental sensing — the system's eyes and ears

IMD Open Data API

- Rainfall classification (heavy / very heavy / extremely heavy), heatwave alerts, flood advisories.
- Official source; thresholds map directly to IMD standards written into policy contracts.

CPCB National AQI Feed

- Real-time AQI readings and category classification for pollution trigger evaluation.
- Only authoritative source for CPCB-compliant AQI bands (300 / 400 / 500 thresholds).

OpenWeatherMap API

- 15-minute interval rainfall and temperature polling; fills gaps in IMD data latency.
- Free tier sufficient; polling interval matches the Monitor Agent's event detection cycle.

Google Maps Traffic API

- Congestion index and route feasibility signals for social disruption triggers.
- Used in Composite Disruption Score; detects zone-level blockage without platform data.

OpenStreetMap + Shapely

- Pin code polygon zone mapping; translates IMD district coordinates to worker delivery zones.
- Free, no rate limits; Shapely handles point-in-polygon efficiently for zone eligibility at scale.

Fallback Mechanism: If any external API is unavailable, the system falls back to the last known values cached in Redis (TTL: 30 minutes). Workers are not denied coverage due to third-party API downtime.

LAYER 2 | AI / ML Intelligence Layer

Feature engineering, risk scoring, dynamic pricing, and fraud detection

Feature Engineering:

Pandas Feature Pipelines

- Structured feature construction from raw API data — rainfall rolling windows, AQI delta, zone congestion index, worker earnings percentile, and tenure features. All ML models consume these standardised feature vectors.
- Centralised pipeline keeps data contracts explicit; features are unit-tested independently of model logic.

Predictive Models

Random Forest Regressor

- Income Prediction Model — estimates expected daily / weekly earnings from platform features.
- Handles non-linear gig income patterns; robust to surge outliers; scikit-learn, CPU-native, no GPU required.

XGBoost Classifier

- Disruption Risk Prediction — zone-specific weekly probability across all trigger types.
- Industry benchmark for structured insurance data; captures weather × traffic × zone feature interactions.

Ridge Regression

- Premium Adjustment Model — maps expected loss to personalised weekly premium.
- Fully explainable signed coefficients; handles correlated risk features; one-line audit trail per worker.

Fraud Detection:

PyTorch LSTM (Tiny)

- Behavioural Intelligence — 10-minute time-series fraud detection; 25K parameters, lightweight CPU inference suitable for real-time evaluation.
- Only sequential model that captures the rhythm difference between a moving rider and a static spoofer.

NetworkX + Louvain Community Detection

- Relational Intelligence — claim graph construction and community detection for ring fraud at MVP scale (~400 escalated claims/week).
- Phase 1 ring detection; trivial compute at this claim volume; graph structure exposes coordinated Telegram-style syndicates.

PyTorch Geometric GCN [FUTURE SCOPE]

- Phase 3 upgrade — learned edge weights for improved ring detection on novel syndicate patterns.
- Supervised on labelled ring data collected in Phase 1–2; better sensitivity than heuristic edges. Excluded from MVP to keep scope realistic.

Explainability & Audit:

SHAP

- Post-hoc explainability on LSTM fraud scores — feature attribution per held claim for auditors.
- Regulatory and audit requirement; turns an opaque score into a human-readable one-line explanation.

Claude Haiku (LLM Audit Assistant)

- Optional audit layer — RAG-augmented edge case documentation for the ~1% of claims that pass all ML stages. Generates structured reasoning notes for human reviewers.
- RAG over policy docs and event logs prevents hallucination. LLM is not in the critical claim path; it assists auditors, not replaces them.

LAYER 3 | Agent Orchestration Layer

Four stateful agents — adaptive decisions, not hardcoded pipelines

Each agent runs as an async Celery worker, sharing state through Redis. Agents are used where state must be held across multiple poll cycles or where multi-step lifecycle management (retries,

timers, circuit breakers) is required. All agents are independently testable with mocked Redis state.

Monitor Agent

- Polls IMD and CPCB every 15 minutes; tracks cumulative zone rainfall; fires trigger events when thresholds are crossed.
- Holds rolling 24-hour rainfall state across polls — a stateless function cannot accumulate this without repeated DB calls.

Fraud Agent

- Cascade escalation with cluster-context routing; manages 48-hour hold timers and auto-release.
- Cluster-context escalation of sub-threshold workers requires joint state that no static rule can hold.

Payout Agent

- UPI batch submission, retry logic on failed transactions, circuit breaker at 80% pool draw.
- Stateful multi-step payout lifecycle with timers and failure recovery — impossible as a pure function.

Insight Agent

- **Periodic retraining module — labels confirmed fraud cases, updates model performance, and generates dashboard metrics.**
- **In the MVP, retraining is performed via batch runs on collected data. The architecture supports automated periodic retraining in production.**

LAYER 4 | Backend & Data Layer

compute, storage, security, and observability

Compute

Celery + Redis (Message Broker)

- Agent task queue — all four agents run as async Celery workers sharing state through Redis pub/sub.
- Decouples agents from the API; Celery handles retries and scheduling; Redis pub/sub avoids tight coupling.

Storage

PostgreSQL + PostGIS

- Primary relational store — worker registry, policies, zone maps, claims, audit logs. ACID compliance for financial records; PostGIS handles zone polygon storage and spatial queries.

Redis (Cache + State Store)

- Monitor Agent zone state, fraud score cache, claim queue, 48-hour hold timers, and API fallback cache.
- Sub-millisecond reads for real-time cascade; TTL-based timers without a per-claim scheduled job.

Security

JWT Authentication

- Stateless token-based auth for all API endpoints — worker, insurer, and admin roles.
- Standard, auditable; tokens carry role claims for RBAC; no session state required in the API tier.

Rate Limiting (SlowAPI / FastAPI middleware)

- Per-IP and per-worker rate limits on registration, claim submission, and payout query endpoints.
- Prevents automated abuse of claim endpoints; configurable per endpoint without modifying business logic.

Logging & Observability

Loguru (Structured Logging)

- Structured JSON log output for all agent actions, API requests, trigger events, fraud decisions, and payout outcomes.
- Machine-readable logs enable post-hoc audit of every claim decision; log level configurable per environment.

Fraud Audit Trail

- Every fraud escalation, hold, release, and override is written to a dedicated audit_log table in PostgreSQL with agent ID, timestamp, SHAP explanation, and decision.
- Regulatory requirement; supports both automated review by the Insight Agent and manual review by human auditors.

LAYER 5 | Payment & Payout Layer

Instant income replacement — the moment that matters most

Razorpay Payout API (Sandbox)

- Bulk UPI transfer simulation — batch payouts to worker UPI IDs within minutes of trigger confirmation.
- Sandbox fully functional; realistic payout flow demo without live funds; supports bulk contacts.

UPI Deep Link (Simulator)

- Worker-facing payment notification — push confirmation with payout amount and trigger reason.
- UPI is the dominant payment rail for gig workers in India; zero friction for recipients across all Android devices.

LAYER 6 | Frontend & Visualisation Layer

Worker dashboard + Insurer analytics — two audiences, two views

React (Web, Mobile-Responsive)

Worker interface — onboarding, plan selection, policy status, payout history, claim tracker, and audit chatbot. Component reuse across worker and admin views; mobile-first layout optimised for low-end Android devices.

Recharts

- Insurer analytics dashboard — loss ratio trends, zone risk heatmap, claim volume, fraud hold rate.

Mapbox GL JS

- Live zone risk map — disruption event overlays, affected zone polygons, active trigger view.

LAYER 7 | Infrastructure & Deployment Layer

Containerised, CI/CD-ready, free-tier deployable

Docker + Docker Compose

- Single-command local spin-up of all services — FastAPI, PostgreSQL, Redis, Celery workers, and React frontend.

GitHub Actions (CI/CD)

- Automated lint, test, and build pipeline on every commit; deploys to Render on merge to main.

Render / Railway (Cloud Hosting)

- Free-tier cloud hosting for FastAPI + PostgreSQL; Celery agents as background services.

10.1 Technology Decision Rationale

Every technology was chosen for a specific reason tied to the system's constraints: CPU-first inference, free-tier APIs, 30-day build timeline, and India's gig payment context.

Decision	Alternative Considered	Why We Chose It	What It Enables
PyTorch LSTM over Isolation Forest	Isolation Forest	Temporal fraud signal is in the rhythm, not the snapshot	Distinguishes genuine rider from static spoofer
NetworkX + Louvain over XGBoost	XGBoost (row-independent)	Ring fraud is a graph property invisible to per-row models	Catches coordinated Telegram-style syndicate attacks
Ridge Regression for pricing	Neural network pricing	Regulators require explainable coefficients	One-line premium audit trail per worker
Celery agents over hardcoded pipeline	FastAPI background tasks	Agents hold state; handle data delays adaptively	Monitor Agent accumulates 24-hr rainfall without repeated DB calls
OpenStreetMap over Google Maps	Google Maps (costly)	Free polygon data; Shapely handles zone mapping offline	Zero cost for zone eligibility at 100K worker scale
RAG-augmented LLM over fine-tuned	Fine-tuned domain model	Policy docs change weekly; RAG reads latest context at runtime	Edge case decisions stay current without retraining
JWT + Rate Limiting	Session auth / no limits	Stateless, auditable; prevents claim endpoint abuse	Secure multi-role API without session state overhead

Decision	Alternative Considered	Why We Chose It	What It Enables
Loguru structured logging	print() / ad-hoc logging	Machine-readable JSON enables audit queries over decision logs	Regulatory-grade fraud audit trail from day one
Redis fallback cache	Hard fail on API timeout	Workers must not lose coverage due to third-party downtime	Resilient trigger evaluation under partial API outage
Pandas feature pipelines	Ad-hoc preprocessing in notebooks	Centralised, tested pipelines enforce data contracts across all models	Feature consistency between training and inference environments

10.2 Weekly Inference Economics

The cascade ensures compute cost scales with suspicion level, not claim volume. Across 10,000 weekly claims, total inference spend is under $2 USD.

Stage	Technology	Claims Handled	Cost / Claim	Weekly Total (10K)
Stage 1	Rule engine (Python)	~8,000 (80%)	~0 ms / $0	~$0
Stage 2	Tiny LSTM (PyTorch CPU)	~1,500 (15%)	~5 ms / $0	~$0 (CPU only)
Stage 3	NetworkX + Louvain (CPU)	~400 (4%)	~50 ms / $0	~$0 (CPU only)
Stage 4	LLM API (Claude Haiku)	~100 (1%)	~$0.01	~$1.00 / week
Total	Full cascade	10,000 claims	—	< $2 USD / week

Justification for our choosing for Web Platform:

When we started building BHIMA ASTRA, one question stayed with us throughout the process:

how do we make this solution actually reachable to the people who need it the most?

Gig delivery workers do not operate in a controlled environment. They are constantly moving, switching locations, working across different platforms, and using a wide range of devices. Building something that depends on a specific device type or installation requirement would immediately create barriers.

That is why we chose web development.

A web-based system allows instant access without requiring downloads, updates, or storage space. A worker can open the platform on any device, at any time, and immediately interact with their policy, payouts, and data. This simplicity is important because in real-world scenarios, even small friction points can lead to drop-offs.

Another important reason was scalability. Our system depends on real-time data ingestion, AI-driven processing, and continuous monitoring of environmental signals. A web architecture allows us to centralize these operations while ensuring that updates are reflected instantly for all users without requiring any manual intervention from their side.

We also considered integration. BHIMA ASTRA is designed to work alongside multiple Q-commerce platforms. A web-based approach makes it easier to integrate APIs, dashboards, and partner systems without being tied to a single ecosystem.

Most importantly, web development allowed us to design for inclusivity. The goal was not just to build a technically strong system, but to ensure that it is usable by someone who may not be highly tech-savvy. A simple interface, accessible through a browser, reduces complexity and makes the system feel approachable.

In the end, choosing web development was not just a technical decision. It was a decision aligned with accessibility, scalability, and real-world usability. It ensures that the solution is not only built well, but also used.

14. Final Conclusion

India's gig delivery workforce is one of the fastest-growing labour segments in the world, yet it operates without any structured income protection. When rainfall floods a city, temperatures spike during a heatwave, or air quality reaches hazardous levels, millions of workers simply lose income with no recourse, no savings, and no safety net.

BHIMA ASTRA changes that. By combining AI-driven risk pricing with fully automated parametric payouts, it delivers income protection that is immediate, affordable, and scalable: matching the income rhythm of weekly gig work rather than the annual cycles of traditional insurance.

The model succeeds across all three stakeholder dimensions simultaneously. Workers receive automatic payouts without filing a claim. Platforms gain a welfare benefit that aids retention and regulatory compliance at zero premium cost. Insurers gain a high-frequency, low-overhead revenue stream with actuarially sustainable loss ratios.

Five design differentiators, including cross-platform AI training, multi-level severity bands, manager-as-intelligence for social disruptions, policy portability, and composite disruption scoring, directly address the core failure modes of parametric insurance: basis risk, single-signal brittleness, and platform dependency [1]-[12].

The fraud architecture ensures that the automated payout model is not exploitable at scale. By resolving approximately 80% of claims instantly at the rule layer while deploying adaptive behavioral and graph-based detection for the remainder, the system protects the liquidity pool without penalising genuine workers [22]-[31].

Phase 1 pilot projections demonstrate financial viability from as few as 2,000-2,500 policyholders, with loss ratios converging to the 50-65% actuarial target as the risk pool scales. The path from pilot to Rs. 39 Crore in annual premiums across three platforms and 100,000 workers is a realistic, evidence-based roadmap.

Closing Statement

This is not a proof of concept. It is a deployable, actuarially grounded, and research-backed parametric income protection system. Built for the 23.5 million gig workers who will be delivering for India's Q-commerce economy by 2030, and who currently have nothing.

Appendix A: Dataset & Variables Design

The dataset is designed to model income variability of gig delivery workers and quantify income loss resulting from measurable external disruptions such as weather conditions, environmental factors, and operational events.

A.1 Dataset Objective

The dataset integrates platform-specific earning structures, worker productivity patterns, environmental risk indicators, and operational and contextual variables. This enables income prediction, disruption impact estimation, parametric claim triggering, and AI-driven risk modeling.

A.2 Variable Classification Principle

All variables are classified into three categories:

(1) Observed Variables — directly obtained from platform ecosystem data;

(2) External Variables — sourced from real-world APIs (weather, AQI, traffic);

(3) Derived Variables — computed for insurance and analytical purposes.

This classification ensures traceability, auditability, and alignment with real-world insurance data practices.

A.3 Worker Profile Variables

Variable	Description	Range	Type	Source
worker_id	Unique identifier	categorical	observed	system
platform	Delivery platform	7 platforms	observed	[1]
city	Operating location	Tier-1 / Tier-2	observed	[1]
vehicle_type	Delivery vehicle	bike / cycle	assumed	industry

Variable	Description	Range	Type	Source
shift_hours	Daily working hours	4–18 hrs	observed	[2][3]
experience_level	Years of experience	0–9 yrs	observed	[2]
employment_type	Full-time / Part-time	categorical	derived	[2][3]

A.4 Productivity & Earnings Variables

Variable	Description	Range	Type	Source
orders_per_day	Deliveries per day	20–55	observed	[1]
orders_per_hour	Productivity rate	2–5	derived	[1]
delivery_distance	Distance per order	0.7–5 km	observed	[1]
earnings_per_order	Earnings per delivery	₹25–₹75	observed	[1]
daily_income	Daily earnings	₹560–₹3,000	observed	[1]
weekly_income	Weekly earnings	₹4,000–₹21,000	observed	[4]
monthly_income	Monthly earnings	₹14K–₹29K	observed	[1]
base_pay	Base earning per order	—	observed	core income
surge_multiplier	Surge pricing factor	—	observed	demand/weather
incentive_bonus	Incentive earnings	—	observed	delivery targets
tip_amount	Customer tips	—	observed	supplementary

A.5 Environmental Variables

Variable	Description	Range	Type	Source
rainfall	Rainfall intensity	0–300 mm/day	observed	[7]
rainfall_category	Light/Moderate/Heavy/Extreme	categorical	derived	[7]
temperature	Ambient temperature	20–48°C	observed	[8]
heat_index	Feels-like temperature	25–55°C	derived	[8]
AQI	Air Quality Index	0–500	observed	[9]
AQI_category	Good → Hazardous	categorical	derived	[9]
traffic_index	Congestion level	0–100	observed	[10]
flood_alert	Flood occurrence flag	0 / 1	observed	[7]

Variable	Description	Range	Type	Source
wind_speed	Wind intensity	—	observed	delivery safety
visibility	Road visibility conditions	—	observed	delivery efficiency
road_closure_flag	Road accessibility	binary	observed	disruption severity
weather_severity_index	Composite weather score	—	derived	predictive modeling

A.6 Event Variables

Variable	Description	Type	Source
platform_outage	App downtime	binary	[1]
zone_shutdown	Area restriction	binary	[1]
curfew_flag	Government restriction	binary	external
strike_flag	Worker/platform strike	binary	external
order_demand_drop	Reduction in order volume	derived	primary income driver
supply_constraint	Worker availability constraints	derived	order allocation
delivery_delay	Average delay per order	derived	effective productivity

A.7 Insurance & Risk Variables

Variable	Description	Range	Type
expected_income	Predicted daily income	₹560–₹3,000	derived
actual_income	Observed income	₹0–₹3,000	observed
income_loss	Income reduction	₹0–₹2,000	derived
disruption_flag	Event occurrence indicator	0 / 1	derived
coverage_limit	Maximum payout per event	₹200–₹1,000	design
weekly_premium	Weekly policy price	₹30–₹80	design
payout	Claim amount	₹0–₹1,200	derived
risk_score	Predicted disruption risk	—	derived
claim_probability	Probability of payout	—	derived
historical_loss_rate	Historical loss frequency	—	derived

Variable	Description	Range	Type
zone_risk_level	Risk classification (low/medium/high)	—	derived

Key Insight: Gig worker income is stochastic and highly sensitive to external environmental and operational factors, making it suitable for parametric insurance based on objective, real-time measurable triggers.

References

All references below are Q-commerce and directly related research used in this document.

Index	Paper Name	Reference Details
[1]	[Swiggy Instamart Delivery Partner Ecosystem]	Swiggy Instamart Delivery Partner Ecosystem (Internal Research Document)
[2]	[BigBasket Now Delivery Partner Ecosystem]	BigBasket Now Delivery Partner Ecosystem (Internal Research Document)
[3]	[Flipkart Minutes Delivery Partner Ecosystem]	Flipkart Minutes Delivery Partner Ecosystem (Internal Research Document)
[4]	[Amazon Now Delivery Partner Ecosystem]	Amazon Now Delivery Partner Ecosystem (Internal Research Document)
[5]	[FreshToHome Express Delivery Partner Ecosystem]	FreshToHome Express Delivery Partner Ecosystem (Internal Research Document)
[6]	[Zepto Delivery Partner Ecosystem]	Zepto Delivery Partner Ecosystem (Internal Research Document)
[7]	[Blinkit Delivery Partner Ecosystem]	Blinkit Delivery Partner Ecosystem (Internal Research Document)
[8]	[IMD Rainfall Classification and Heatwave Criteria]	India Meteorological Department (IMD). Rainfall Classification and Heatwave Criteria. https://mausam.imd.gov.in/
[9]	[CPCB National AQI Classification Standard]	Central Pollution Control Board (CPCB). National AQI Classification Standard. https://cpcb.nic.in/
[10]	[Curfews Protests and Social Unrest Forecasting]	Redl, C., and Hlatshwayo, S. (2021). Forecasting Social Unrest: A Machine Learning Approach. IMF Working Papers, 2021/263.
[11]	[Zepto GPA Group Insurance and Platform Insurance]	PARAMETERS.docx: Zepto GPA Group Insurance Model (Internal); CompareC over. https://comparecover.in/
[12]	[GPS and Composite Disruption Score Design]	GPS and Data Flow Design Document: Composite Disruption Score (Internal Research Document)
[13]	[Random Forests]	Breiman, L. (2001). Random Forests. Machine Learning, 45, 5-32. https://doi.org/10.1023/A:1010933404324
[14]	[Predicting Salaries with Random-Forest Regression]	Eichinger, F., and Mayer, R. (2022). Predicting Salaries with Random-Forest Regression. In Data Science Analytics and Applications. Springer.

Index	Paper Name	Reference Details
[15]	[Supervised Machine Learning for Alumni Income]	Galar, C., et al. (2022). Supervised Machine Learning Predictive Analytics for Alumni Income. Journal of Big Data, 9(1).
[16]	[XGBoost: A Scalable Tree Boosting System]	Chen, T., and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proc. 22nd ACM SIGKDD, 785-794. https://doi.org/10.1145/2939672.2939785
[17]	[XGBoost for Insurance Claim Prediction]	Fauzan, M. A., and Murfi, H. (2018). The Accuracy of XGBoost for Insurance Claim Prediction. IJASCA, 10(2).
[18]	[Google Maps Platform Routes and Traffic API]	Google Maps Platform. Routes and Traffic API Documentation. https://developers.google.com/maps/documentation/routes
[19]	[AI-Driven Dynamic Premium Pricing in Insurance]	AccelTree (2025). AI-Driven Dynamic Premium Pricing in Insurance. https://acceltree.com/white-papers/
[20]	[Dynamic Pricing Gives Insurers a Competitive Edge]	Insurance Thought Leadership (2024). Dynamic Pricing Gives Insurers a Competitive Edge. https://www.insurancethoughtleadership.com/
[21]	[Dynamic Pricing in Insurance Using AI]	SimpleSolve (2025). Dynamic Pricing in Insurance Using AI and Predictive Analytics. https://www.simplesolve.com/blog/
[22]	[Intelligent Financial Fraud Detection]	West, J., and Bhattacharya, M. (2016). Intelligent Financial Fraud Detection: A Comprehensive Review. Computers and Security, 57, 47-66.
[23]	[Isolation Forest]	Liu, F. T., Ting, K. M., and Zhou, Z. H. (2008). Isolation Forest. IEEE ICDM, 413-422. https://doi.org/10.1109/ICDM.2008.17
[24]	[Graph-Based Anomaly Detection for Fraud]	Pourhabibi, T., et al. (2020). Fraud Detection: A Systematic Literature Review of Graph-Based Anomaly Detection. Decision Support Systems, 133, 113303.
[25]	[XGBoost for Insurance Claim Prediction (Fauzan)]	Fauzan, M. A., and Murfi, H. (2018). The Accuracy of XGBoost for Insurance Claim Prediction. IJASCA, 10(2).
[26]	[Long Short-Term Memory]	Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. Neural Computation, 9(8), 1735-1780. https://doi.org/10.1162/neco.1997.9.8.1735
[27]	[AI-Driven Dynamic Premium Pricing (AccelTree)]	AccelTree (2025). AI-Driven Dynamic Premium Pricing in Insurance (White Paper).
[28]	[Semi-Supervised Graph Attentive Network for Fraud]	Wang, D., et al. (2019). A Semi-Supervised Graph Attentive Network for Financial Fraud Detection. IEEE ICDM, 598-607.

Index	Paper Name	Reference Details
[29]	[Fast Unfolding of Communities in Large Networks]	Blondel, V. D., et al. (2008). Fast Unfolding of Communities in Large Networks. J. Stat. Mech., P10008.
[30]	[Semi-Supervised Classification with GCNs]	Kipf, T. N., and Welling, M. (2017). Semi-Supervised Classification with Graph Convolutional Networks. ICLR. https://arxiv.org/abs/1609.02907
[31]	[SHAP: A Unified Approach to Interpreting Model Predictions]	Lundberg, S. M., and Lee, S. I. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS, 30. https://arxiv.org/abs/1705.07874
[32]	[Economic Lives of Digital Platform Gig Workers India]	Brailovskaya, V., et al. (2023). Economic Lives of Digital Platform Gig Workers: India. IDinsight. https://www.idinsight.org/
[33]	[CGAP on Embedded Insurance Adoption]	CGAP. Embedded Insurance Adoption and Bancassurance Models. (Internal Reference)
[34]	[World Bank Stories of Climate Resilience Jan Sahas]	World Bank (2021). Stories of Climate Resilience: Jan Sahas parametric pilot, India.
[35]	[Future of India's Gig Work]	Drishti IAS (2024). Future of India's Gig Work. https://www.drishtiias.com/
[36]	[Rajasthan Platform-Based Gig Workers Act 2023]	Rajasthan Platform-Based Gig Workers Act, 2023. https://prsindia.org/
[37]	[Karnataka Gig Workers Bill 2024]	Karnataka Gig Workers Bill, 2024; Union Budget 2025.
[38]	[Swiggy Accident Cover and Welfare Programs]	Rathi, A., et al. Understanding Food Delivery Platform: Delivery Persons' Perspective. TISS Hyderabad. (Internal Research)